



Charte de Travail

17/12/2025

Membres

AIT MOHAMED Younes (aitmohay)

BEAURAIN Tom (beurato)

BEN ISMAIL Wajd (benismaw)

FURLANI DA SILVA SOARES Manuela (furlanim)

MARIA ENGLER PINTO Carlos (mariaenc)

Objectif de la charte

Nous définissons cette charte pour garantir une organisation efficace, une communication claire et un cadre commun de travail (qualité, respect des délais, résolution de désaccords). Elle sert de référence en cas de difficultés ou de conflit.

Compétences de l'équipe

Objectif pour améliorer

Renforcer la qualité (tests + CI), améliorer l'architecture et la génération de code. Progresser en Git et intégration, et améliorer en tests et debug.

Compétences des membres de l'équipe

- Programmation en Java (bases solides à avancées)
- Connaissances en algorithmique et structures de données
- Notions de compilation (lexing, parsing, AST)
- Utilisation d'outils de développement (Git, Maven)
- Capacité à lire et comprendre de la documentation technique

Les compétences sont réparties de manière hétérogène, ce qui permet l'entraide et la montée en compétences de chacun.

SWOT de l'équipe (synthèse)

Forces

- Motivation et implication régulière
-

- Bonne capacité à se répartir les tâches
- Base solide en outils (Git/GitLab) et développement

Faiblesses

- Expérience limitée sur certains aspects spécifiques (génération de code, chaîne complète)
- Risque de conflits d'intégration si le workflow n'est pas strict

Opportunités

- Renforcer nos pratiques d'ingénierie logicielle (architecture, tests, CI)
- Extension technique progressive et valorisable

Menaces

- Contraintes de temps (autres cours/rendus)
- Découverte tardive de bugs si les tests ne sont pas mis en place tôt

Valeurs communes (attendus / tolérés / proscrits)

Nous adoptons des valeurs communes pour éviter les malentendus et maintenir une dynamique saine tout au long du projet. Nous attendons de chacun une communication transparente : tout blocage, retard ou difficulté doit être signalé rapidement afin de pouvoir ajuster le planning et aider la personne concernée.

Nous valorisons le respect mutuel : les discussions techniques doivent rester centrées sur les idées et le code, jamais sur les personnes. Nous privilégions également la qualité : nous cherchons à produire un code lisible, maintenable et vérifié par des tests, même lorsque le rythme est soutenu.

Rôles et responsabilités

Chaque tâche a un responsable (owner) et au moins un relecteur (reviewer), mais on peut faire évoluer les rôles de chacun afin que chacun ait la possibilité d'occuper différents postes. Les rôles fixés jusqu'au Suivi 2, puis rotation possible.

Chef(fe) de projet / facilitateur

- Anime les réunions, suit le planning, arbitre en cas de blocage
- Assure le lien avec l'enseignant (questions, points d'avancement)

Référent intégration / CI

- Maintient la stabilité de la branche principale

- Surveille les pipelines et gère les conflits de merge

Référent qualité / tests

- Définit la stratégie de tests (unitaires + intégration)
- Vérifie que les tâches livrées sont testées et reproductibles

Référent architecture

- Propose l'organisation des packages et conventions de code
- Coordonne les évolutions structurantes (refactorings)

Référent documentation

- Maintient README et documents internes (décisions, procédures)

Organisation du travail (process)

Nous organisons notre travail en sprints courts correspondant à un suivi, c'est-à-dire un par semaine avec des objectifs clairs et mesurables. Les tâches sont formalisées sous forme d'issues et priorisées au début de chaque sprint, avec un responsable identifié pour chaque tâche.

Nous utilisons un workflow Git basé sur des branches par fonctionnalité/correction et des merge requests systématiques, afin d'assurer une intégration progressive et contrôlée. La branche principale doit rester stable et compilable à tout moment : une fonctionnalité n'est intégrée que si elle passe la compilation, les tests automatisés et une relecture.

Afin d'améliorer la qualité du code et de limiter les risques d'erreurs d'intégration, l'équipe adopte une organisation circulaire des tâches.

Pour chaque fonctionnalité ou correction, les rôles sont répartis de manière complémentaire et peuvent tourner entre les membres :

- Un premier membre développe la fonctionnalité (codage).
- Un second membre est chargé de tester, valider et éventuellement corriger la fonctionnalité avant toute intégration.

En parallèle, un autre binôme peut travailler sur une fonctionnalité différente en suivant le même principe (développement + tests).

- Le workflow Git est organisé en plusieurs niveaux :
- Les développements sont effectués sur des branches dédiées.
- Les premiers commits sont poussés sur un dépôt ou une branche de validation (branche « réserve »), servant de zone tampon pour les tests et vérifications.

- La branche principale (master/main) est considérée comme officielle et stable : un merge n'est autorisé que lorsque la fonctionnalité est entièrement testée, validée par un relecteur, et que l'ensemble du projet compile et fonctionne correctement.

Communication au sein de l'équipe

Canaux

- Messages rapides : WhatsApp
- Planning: Clickup
- Documentation : dossier [/doc](#) + README dans le dépôt

Rendez-vous

Une réunion par jour, du lundi au vendredi (en ligne ou en présentiel), dont au moins une en présentiel.

Gestion des tensions / conflits

- On discute d'abord du problème avec des faits concrets (bug, délai, qualité, charge)
- Si désaccord technique : arguments + décision tracée dans une issue+sollicitation de l'enseignant.
-